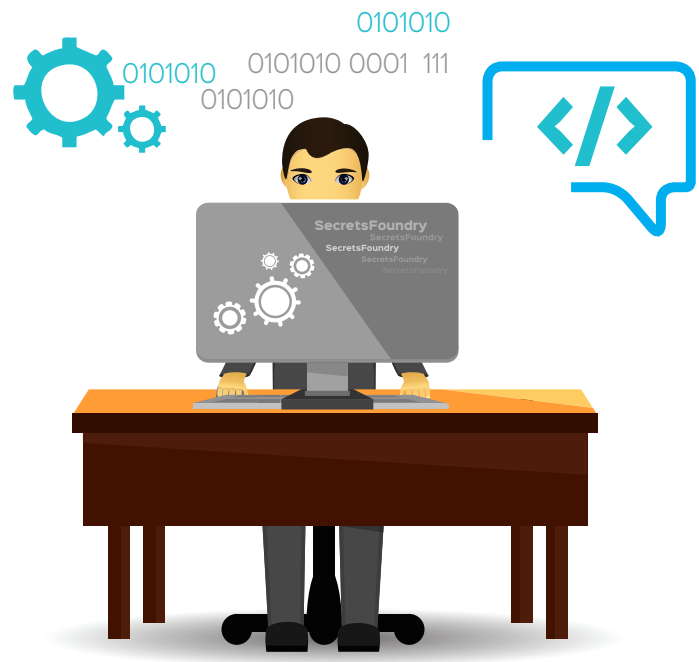
Managing Secrets with SecretsFoundry



While building an app, you need to manage environment variables and preserve their credentials. SecretsFoundry helps you to automatically fetch these variables from different secret managers, helping to make configurations across environments seamless.

One of the biggest challenges in application development is managing environment variables and preserving their credentials. Let's go over different approaches in detail. Imagine you are building a simple app that talks to a MongoDB database through a connection and starts serving on port 3000. As easy as it sounds, it is very wrong to hard-code credentials inside the code, as it will then run only on the specified system where it has been coded. Even if shared with fellow engineers, the code will not be able to connect with the MongoDB instance or run in different deployed environments.

```
server = Server()
mongoDB = new MongoDb("mongodb://username: password@
locahost:3000/testdb")
server.get('/users', function() {
mongoDB.getUsers()
})
server.start(port=3000)
```

While cross-environment configurations are pain points, following the 12 factor app methodology helps in separating them from the code. Your configuration comprises credentials that talk to the backend services (SQL, MongoDB or Memcached). There can also be credentials that talk to different APIs if the code is interacting with Twitter or Amazon S3 APIs. There can be host names or ports that can vary across deployments, but this should be a part of the configuration.

What is 12 factor app methodology?

- Separate configuration from code
- Application's configuration is everything that varies between deploys
  - Resource handles to the database, Memcached and other services
  - Credentials to external services such as Amazon S3 or Twitter
  - Per-deploy values such as the canonical host name for the deploy